Lecture 1 · Part I · Lecture notes

What machine learning is, and how we will work

Dataset: California housing

How to use these notes

The primary text for this lecture is Géron, Chapters 1–2. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 What the course is about

The course has a one-sentence description: how to build a machine learning system, and how to know whether it works. The second clause is the hard one. Fitting a model is a few lines of library code, and it has been for a decade. Establishing that the number the model reports means what it appears to mean is the whole of the rest of the course, and it is the part that does not get easier with better libraries.

1.1 Why the second clause is hard

A bug in a training loop does not crash. It returns a plausible number. That sentence is the reason this course exists in the shape it does, and it is worth five concrete instances before we go any further:

- a scaler fitted before the train/test split — information leaks from the test set into the training statistics, the score improves, and the system is broken in deployment;
- a missing `model.eval()` — dropout stays active while you evaluate, so the reported accuracy is of a model nobody will ever ship;
- a missing `optimizer.zero_grad()` — gradients accumulate across steps, and nothing anywhere raises an error;
- a metric averaged per batch rather than over the set — wrong by construction whenever the batches are not the same size;
- accuracy on an imbalanced set — 90% from a model that has learned to predict one class.

The property that organises this course

Every one of those five runs to completion. Every one of them prints a number. Not one raises an exception, and four of the five produce a number that is *better* than the honest one, which is precisely the direction in which you are least likely to go looking for a bug.

Compilers catch type errors and test suites catch crashes. Neither catches a number that is merely wrong, and machine learning produces those in quantity. What catches them is evidence: a baseline beside the number, an ablation showing the number moves when you remove the component that supposedly earned it, and a stated failure case. Those three are the working habits this course is trying to install, and they are examined directly.

1.2 What kind of problem needs learning at all

A learning system is worth building when the rule you would otherwise write by hand is long, unstable, or unknown. Spam filtering is the standard example: the hand-written version is a list of patterns that its adversary edits around every week, so the maintenance never ends. A learning system replaces the list with a procedure for deriving the list, and the maintenance becomes retraining.

Three consequences follow, and they are worth stating before any method:

- **If a short exact rule exists, write it.** A learning system is strictly worse than `if amount > limit` when that is genuinely the rule: it is slower, larger, harder to audit, and occasionally wrong.
- **The system is only as good as what it learns from.** There is no method in this course that repairs data which does not contain the answer, and a great deal of the course is about noticing that case.
- **It will fail quietly.** See above. This is why the evaluation material is not an appendix.

1.3 The taxonomy, briefly

The distinctions are standard, and the reason to have them straight is that they determine which methods are even candidates.

Supervision	What the training data carries
supervised	every instance carries its expected output
unsupervised	no labels; structure has to come from the inputs alone
semi-supervised	a few labelled instances among many unlabelled
self-supervised	labels manufactured from the data itself — mask a word, predict it
reinforcement	no labels; a reward signal for a sequence of actions

Cutting a different way: **instance-based** learning keeps the training examples and answers a new query by similarity to them; **model-based** learning fits parameters and discards the examples. And cutting a third way: **batch** learning trains once on everything held in memory, **online** learning consumes a stream and updates as it goes.

1.4 The failure modes, named up front

Four, and every one of them will reappear with a measurement attached:

- **Insufficient data.** The method is not the bottleneck.
- **Unrepresentative data.** The training set is not a sample of the population the system will meet. This is the one that is invisible in every metric you compute on your own data, and §5 is about buying insurance against it.
- **Overfitting.** The model has learned the training set, including its noise. Low training error, high error on anything else.
- **Underfitting.** The model is too rigid to represent the pattern that is there. Both errors high.

The only way to distinguish the last two — indeed the only way to know how a system generalises at all — is to try it on cases it has never seen. Hence a training set and a test set, hence the rule that the test set is spent once, and hence the fact that the split happens before the exploration rather than after it.

2 The working method

You will write machine learning code with an assistant: if not in this course, then in the thesis after it and the job after that. This course is explicit about how, rather than pretending otherwise, and it spends the first half of Lecture 1 on it because the alternative is spending the rest of the course undoing the habits.

The argument for spending the time is short. Applied machine learning is a small amount of library glue and an enormous amount of data work and evaluation. The scarce skill is judgement about whether a result can be trusted. An assistant supplies the glue and none of the judgement.

2.1 The four rules

Four rules for AI-assisted work

1. **Never keep code you cannot explain line by line.**
2. **Every number needs a baseline.** A metric with nothing to compare it to is decoration.
3. **Every model needs an ablation.** Remove a component; show the number moves.
4. **Report the failure cases.** A system with no known failure mode has not been tested.

Rules 2 to 4 are what a result has to come with before anyone should believe it — yours, a colleague's, or an assistant's. They are also, roughly, Part C of the written paper. Rule 1 is Part B: the paper puts a cell in front of you and asks what would have to be true for its number to be trusted.

2.2 The loop, and whose job each step is

#	Step	What it means	Who
1	Specify	state the input, the output, the constraint, the check	you
2	Generate	and then stop	the assistant
3	Read	as a reviewer, not as the author	you
4	Test	against a case whose answer you know	you
5	Verify	that the number means what you think it means	you

Four of the five are yours. The one that is not is the one that was never the hard part. Steps 3 to 5 are, quite literally, the syllabus.

Step 1 — Specify

A usable specification names four things: the **input** (shape, types, provenance), the **output** (exactly what object, with what guarantees), the **constraint** (what must not happen), and the **check** (how you will know it worked). Written out for a task from later in this lecture:

A specification, in full

Task: prepare the housing features for a model.

input — the 16,512-row training frame, 8 numeric columns and 1 categorical, with 168 missing values in `total_bedrooms`
output — a fitted `ColumnTransformer` and the transformed matrix
constraint — the imputer and the scaler are fitted on the training rows *only*; the test rows are transformed with those statistics and never contribute to them
check — the output has 16,512 rows and 13 columns (8 numeric plus 5 one-hot levels) and no `NaN`

The part people omit is the third. Almost every silent failure in machine learning is a missing constraint, not a missing feature — and the constraint above is exactly the one whose violation produces the leaked scaler from the list at the top of these notes.

Every code cell in every notebook in this course is preceded by a box in that form. The box is step 1; the cell below it is what step 2 returned; the check line is step 4, written down in advance. Read the box, answer the check in your head, then run the cell, and you have done the whole loop with only the free step handed to you.

Step 2 — Generate, then stop

The dangerous moment is not the generation. It is the reflex to run it. Generated code is read *before* it is executed, because output you have already seen is very hard to disbelieve. In a domain where wrong code still returns a plausible number, running first destroys the only cheap opportunity to notice.

Step 3 — Read it like a reviewer

Five questions, in this order, every time:

1. **What touched the test set?** Trace every line backwards to its data source.
2. **What was fitted, and on what?** `fit` and `transform` are different verbs.
3. **What is the shape here?** Silent broadcasting is silent.
4. **What was dropped?** Rows, columns, `NaN`s — count them.
5. **What is the default I did not ask for?** Every unnamed argument is a decision someone else made.

Question 5 is not pedantry. Later in this lecture a single unrequested default — a colormap — produces a figure that is colourful, ran without error, and is wrong.

Step 4 — Test against a case you already know

Not "does it run". Does it give the right answer on an input whose answer you can state independently? Four kinds of case, all cheap:

- **A shape.** 16,512 rows in, 16,512 rows out — and if one-hot encoding added five columns, 13 out, not 9.
- **A count.** 168 missing values before, 0 after.
- **A degenerate case.** A constant column has zero variance. What does the scaler do with it?
- **Arithmetic you can do on paper.** A layer with 32 inputs and 32 outputs has $32 \times 32 + 32 = 1{,}056$ parameters. Print the number and compare.

The rule that separates testing from watching

If you cannot state the expected answer *before* running the cell, you are not testing. You are watching.

Step 5 — Verify that the number means what it appears to

The code is correct and the number is still misleading. This is the step nobody does, and it is four questions:

- **Against what?** What does the trivial model score?
- **On which rows?** Training, validation or test — and were they chosen before or after anything was fitted?
- **How much does it move?** A difference smaller than the spread across splits has not been measured.
- **In what units?** An error of 70,000 is meaningless until you know the prices run from 15,000 to 500,000.

Every one of the four is a question about evidence, not about code.

3 The brief

You are a data scientist at a real-estate investment company. The task: use California census data to build a model of housing prices. The data gives population, median income and median housing price for each block group — the smallest geographical unit the US Census publishes, which we will call a **district**. The model predicts a district's median housing price from the other metrics.

3.1 The first question is never technical

What is the business objective? Building a model is not a goal. How the company uses the output determines the framing, the algorithm, the metric, and how much effort is worth spending.

The answer here: the prediction is fed to *another* machine learning system, together with other signals, to decide whether a district is worth investing in. A piece of information fed to a downstream system is called a **signal**, after Shannon.

The consequence of being a component

Nobody downstream ever looks at your prediction. A component that quietly degrades inside a chain of components is not noticed by anyone — which is exactly why the evidence has to come from you, unprompted, rather than from a complaint.

3.2 What the current solution is

District prices are currently estimated manually by experts: a team gathers information and applies complex rules. When they can check their estimates against the truth, they are typically off by more than 30%.

Read that figure carefully, because two things about it matter later. It is a *relative* criterion — 30% of the district's price — and in the next lecture we choose a metric measured in dollars, which is not the same thing. It is also the stakeholder's claim about their own team, not a measurement we made. We will treat it as an assumption and label it as one wherever it appears.

3.3 Framing, in four questions

Question	Answer	Because
Supervision	supervised	every district carries its expected output
Task	multiple, univariate regression	predict one value, from several features
Learning mode	batch, offline	no data stream, and 20,640 rows fit in memory

"Multiple" refers to the several input features; "univariate" to the single output. Predict several values per district and it becomes multivariate regression; add a data stream and it becomes online learning.

The fourth question is the one that can sink the project: *what is everyone assuming?*

Failure condition — the assumption that would have sunk this project

Suppose the downstream system converts our prices into categories — cheap, medium, expensive — and discards the numbers. Then this is a *classification* problem, every regression metric we are about to choose is the wrong one, and we would have built the wrong system competently.
We asked. They need the actual prices. We may proceed — and the value of the question is not that the answer was reassuring, but that it was cheap and the alternative was months.

4 Looking at the data, once

Ten attributes, one row per district. The habit is `head()` then `info()`, always, before anything else.

4.1 What `info()` reports

`RangeIndex: 20640 entries`, ten columns, nine of them `float64` and one `object`. Two facts fall straight out of it.

207 districts are incomplete

`total_bedrooms` has 20,433 non-null values against 20,640 rows. Three options, all legitimate: drop those districts, drop the whole attribute, or fill the gaps with a chosen value. We return to this in the build, and the choice is not obvious — dropping 207 rows is cheap, dropping a column that correlates with the target is not, and filling requires a decision about *what* to fill with and *on which rows the filling value is computed*. That last clause is the constraint from §2.

One attribute is not numerical

`ocean_proximity` takes repeated values from a small set: this is a categorical attribute, not free text.

Category	Districts
`<1H OCEAN`	9,136
`INLAND`	6,551
`NEAR OCEAN`	2,658
`NEAR BAY`	2,290
`ISLAND`	5

Note `ISLAND`: five districts in the whole state. Rare categories cause trouble later — a one-hot column that is nonzero five times in 20,640 rows can appear in a training fold and not in a validation fold, and a stratified split cannot protect a stratum that small.

4.2 Four things the histograms show

Plotting all nine numeric attributes at fifty bins takes one line and repays it immediately.

Observation 1 — the income is not in dollars

`median_income` runs from roughly 0.5 to 15. That is not a salary. The data was scaled and capped: at 15 (in fact 15.0001) at the top and 0.5 (0.4999) at the bottom. The units are roughly tens of thousands of dollars, so 3 means about $30,000. Preprocessed attributes are normal and not a defect; understanding how they were computed is not optional.

Observation 2 — the target is capped

`median_house_value` is capped, and it is our *label*.

Failure condition — why a capped label is more serious than a capped feature

The model will learn that prices never exceed the cap, because in the training data they never do. If the client needs accurate predictions above $500,000, the model is not merely inaccurate there — it is structurally incapable of producing such a number.
Two honest fixes: collect proper labels for the capped districts, or remove them from *both* the training and the test set. Removing them from the training set alone would leave you scoring a model on districts it was forbidden to learn about.

The spike is not small: 992 districts, nearly 5% of the data, carry a label that is a ceiling rather than a price.

Observation 3 — the scales differ wildly

`total_rooms` runs from 2 to 39,320; `median_income` from 0.4999 to 15.0001. Whether that matters depends entirely on the method.

Scaling is required for	Scaling does nothing for
gradient descent, regularised linear models (ridge, lasso), SVMs, k-nearest neighbours, k-means, PCA, neural networks	decision trees, random forests, ordinary least squares

The left column is anything that measures a distance, penalises a coefficient, or takes a step. The right column has reasons: a tree splits on one feature at a time, so any monotone rescaling produces the identical tree; and least squares is equivariant under an invertible affine map of the features — the fit is identical and the coefficients simply absorb the change.

All three models built in Lecture 2 are in the right-hand column, and that is not an incidental remark. It is why the scale-before-split leak, which is the canonical beginner's error, will cost *nothing* measurable here. Lecture 2 measures it over twenty seeds rather than asserting it.

Observation 4 — many attributes are right-skewed

They extend much further to the right of the median than to the left, which makes patterns harder for some algorithms to detect. The usual repair is a transform toward symmetry: a square root or a logarithm. A feature following a power law is a good candidate for log — districts of 10,000 people are about ten times rarer than districts of 1,000, not exponentially rarer, and the log of such a variable is roughly linear in its rank.

5 The split, before the exploration

Four observations from four histograms, and we have not yet plotted one variable against another. That is exactly the moment to stop.

5.1 Data snooping is a bias, not a discipline problem

Your brain is an outstanding pattern detector, which means it overfits. If you study the test set, you will — without any dishonesty, and without being able to point at the moment it happened — choose a model that suits it. Your estimate of generalisation error will be optimistic and you will ship something that underperforms.

The defence is procedural rather than moral: create the test set *before* exploring any further, and do not look at it again. Typically 80/20, though the proportion is a function of size — with ten million instances, holding out 1% is plenty.

5.2 Why random sampling is not automatically enough

Purely random sampling is fine if the dataset is large relative to the number of attributes. Ours may not be, and random sampling then risks **sampling bias**: a test set whose composition differs from the population, so that the error measured on it is an error on the wrong population.

Surveyors do not call 1,000 people at random; they ensure the sample is representative on the dimensions that matter. The experts told us median income predicts price, so the test set must be representative across income levels. That is **stratified sampling**: bin the stratifying variable, then sample within each bin in proportion.

The bins have to be chosen so that no stratum is too small to sample from. Five categories, cut at 1.5, 3.0, 4.5 and 6.0, leave the smallest — category 1 — with 822 districts, which is comfortably enough.

5.3 Does it actually matter? Measure it

The point of this course is that the previous paragraph is a hypothesis until someone counts. Comparing each split's income-category proportions against the full dataset's:

Split	Largest error in stratum proportion
purely random	up to 6.4%
stratified	up to 0.36%

Roughly eighteen times better, for one keyword argument. Note the shape of the argument: the stratification was justified by an expert's claim, and then the claim was checked with a measurement rather than deferred to.

5.4 Assert after every structural step

Three lines, three real bugs

```
assert len(strat_train_set) + len(strat_test_set) == 20640
assert set(strat_train_set.columns) == set(strat_test_set.columns)
assert strat_train_set.index.intersection(strat_test_set.index).empty
```

Nothing was dropped; the two frames have the same columns; no district is in both. Each of the three has been a real bug in a real notebook.

An assertion is a claim about your data that fails *loudly*, and in a domain that otherwise fails silently that makes it the most valuable line in the file. Write one after every split, merge, reshape and drop, and count what you dropped. Reviewer question 4 is then answered by the program instead of by you, every time it runs, including the times you forget to ask.

6 Exploring — the training set, and only that

From here everything is computed on the 16,512-district training split. The other 4,128 are sealed until the last slide of Lecture 2. Work on a copy, so that nothing invented while exploring — a new column, a filter, a dropped row — can reach the split frame by accident, and so that every number below is honestly labelled as a training-set number.

6.1 Look at it, and tune the plot until you can

The data is geographical, so plot latitude against longitude. It looks like California, and beyond that nothing: at full opacity the ink saturates and every district is the same black dot.

Setting `alpha=0.2` changes the picture completely — the Bay Area, Los Angeles, San Diego and the Central Valley appear, because now the density is encoded in the darkness. Same data, one keyword argument. Expect to tune plotting parameters until the pattern shows; a plot that shows nothing is usually a plot, not an absence.

Adding two more channels — radius for population, colour for price — shows that prices relate strongly to location and to population density. Red on the coast, blue inland. The model will not know what "coast" means, but it will learn longitude.

6.2 A default nobody chose, and how to catch it

The three-channel plot above is conventionally drawn with `cmap="jet"`. It ran, it produced something colourful, and it is wrong.

A colour scale is a claim: equal steps in the data look like equal steps on the screen. That claim is testable — plot each colormap's lightness against position along it.

Colormap	Lightness range	Largest reversal	Verdict
`jet`	83.0	1.06	rises, falls, rises
`turbo`	78.9	0.87	better, still not monotone
`viridis`	75.9	0.00	every step is a step

A non-monotone lightness profile invents bands the data does not have and buries detail in the yellow. For the roughly 8% of men with a red–green deficiency, `jet` reverses rank order outright. Photocopy it and it stops being a scale at all.

Failure condition — the shape of this failure

The code ran. Nothing errored. The defect was in a default nobody chose deliberately, and it took a measurement to see. That is every failure in this course, and it is only the first lecture.

6.3 Correlations, and what they cannot see

Pearson's r against the target, on the training split:

Attribute	r with `median_house_value`
`median_income`	0.688380
`total_rooms`	0.137455
`housing_median_age`	0.102175
`households`	0.071426
`total_bedrooms`	0.054635
`population`	−0.020153
`longitude`	−0.050859
`latitude`	−0.139584

Income dominates: one bar is five times the next. That is the feature we already stratified on, on the experts' word, before we had looked — and the measurement says they were right.

These are training-set values. Compute them on all 20,640 rows and the third decimal moves, which is why a number is worth nothing without the set it was measured on attached to it.

Failure condition — two things r does not tell you

r measures *linear* correlation only. A dataset can have $r = 0$ and be completely dependent — any symmetric nonlinear relationship will do it. And $r = \pm 1$ says nothing whatever about the slope: your height in inches correlates 1.0 with your height in nanometres.

6.4 The cap, made visible — and the lines that are not there

Plot the strongest predictor against the target and the $500,001 cap appears as a hard horizontal line: 787 districts in our training split. There are fainter horizontal lines too, and this is where the lecture's method earns its keep. A horizontal stripe means many districts share one exact price. That is a property of the table, not of California, so it is *countable* — and squinting at a scatter plot is not counting.

A well-known description of this dataset names three fainter lines. Counted in our own training split:

Value	Districts	Verdict
\$350,000	62	real — 21× the background
\$450,000	31	marginal
\$280,000	3	indistinguishable from background

A typical price is shared by 3 districts, so the third is not a line at all. Meanwhile the five commonest values below the cap — \$137,500, \$162,500, \$112,500, \$187,500 and \$225,000 — go unmentioned in that description, and they are the strongest stripes in the plot. Every one sits on a multiple of \$12,500, which tells you something about how the survey recorded prices and nothing about the housing market.

The habit, not the numbers

Three values were asserted; one survives. Counting took one line. You will inherit descriptions of data for the rest of your career — from papers, from colleagues, from an assistant — and the cost of checking one against the data is almost always smaller than the cost of building on it.

6.5 Attribute combinations

Raw counts per district are nearly meaningless because districts differ in size. Ratios are not.

Attribute	r with price
`median_income`	0.688
`rooms_per_house`	0.144
`total_rooms`	0.137
`bedrooms_ratio`	−0.256

`bedrooms_ratio` — bedrooms as a fraction of all rooms — is far more informative than either raw count it is built from, and its sign is interpretable: a district whose rooms are mostly bedrooms is a district of small dwellings.

A warning you will need next lecture

When you create combined features, check that they are not too linearly correlated with the existing ones, and in particular avoid simple weighted sums of features already present. **Collinearity** causes real problems for some models — linear regression above all. Lecture 2 derives the normal equation and shows exactly why: the matrix you have to invert becomes singular, or close enough to it that the inverse is numerically meaningless.

7 The notebook

The Lecture 1 notebook runs on Colab's free CPU tier in about two minutes and contains everything above in order: the fetch, `info()`, the missing values, the histograms, the stratified split,

and the exploration. Every code cell carries the specification that would produce it.

What to change

Run it top to bottom once. Then change one thing — the number of income bands in `pd.cut` — and watch the sampling-bias table. Two lines, and it is the whole argument of §5: too few strata and the stratification buys you little; too many and each stratum is too small for its own proportion to be estimated reliably, so the cure starts to resemble the disease.

8 Exercises

Five, in the style of the written paper. Solutions on Lecture 2's deck.

1. A colleague reports that their model reaches an RMSE of $48,000 on the California housing data, and that they chose the model by trying six of them and keeping the best. State, in one sentence, what that number is an estimate of — and what it is not an estimate of. [4]

2. The median income column is capped at 15. Give one consequence for the *model* and one for the *evaluation*, and say which of the two you would raise with the client first. [5]

3. You are given a stratified split on income category and a naive random split of the same data. Both report similar RMSE. Does that mean the stratification was unnecessary? Answer yes or no, with a reason. [4]

4. The brief says the output feeds a downstream system that expects a price. Name the one thing this fact settles about the problem framing, and one thing it leaves open. [4]

5. In the working loop — specify, generate, read, test, verify — exactly one step is done by the assistant. Name it, and say what goes wrong if a student treats a second step as the assistant's. [3]

Summary

Machine learning is worth reaching for when the hand-written rule is long, unstable or unknown, and it fails silently, which is why evidence rather than execution is the standard throughout this course. The working method is five steps of which four are yours: specify with a constraint and a check, read as a reviewer, test against a case whose answer you already know, and verify that the number means what it appears to.

The brief was framed before any code: supervised, multiple univariate regression, batch — and the assumption that would have made it a classification problem was checked with the downstream team rather than guessed. The data was looked at once, in full, which surfaced 207 incomplete districts, a categorical attribute with a five-district level, a target capped at $500,001 for 992 districts, scales spanning four orders of magnitude, and pervasive right skew.

Then the split, *before* the exploration, stratified on the predictor the experts named — and measured at 0.36% worst-case stratum error against 6.4% for a random split. Only then the exploration, on the training set alone: income dominates the correlations at 0.688, a default

colormap turned out to be indefensible under a one-line test, three asserted survey artefacts reduced to one under counting, and a ratio of two weak columns beat both.